Rheinland-Pfälzische Technische Universität Kaiserslautern Landau

Entwurf informationstechnischer Systeme

Implementation
InvisiSpec - Making Speculative Execution Invisible in the Cache Hierarchy

20439: Embedded Systems Project

Tharindu Samarakoon - 430891

Background

- Speculative execution improves performance.
- Speculative loads can become transient instructions.
- Leave observable changes in the cache hierarchy.
  - Enable microarchitectural side channels.
- Attacks leak sensitive data.
  - Meltdown: generate exceptions
  - Spectre: **No exceptions!**
- Fix
  - Meltdown: mark the data loaded by an exception-triggering load as unreadable.
  - Spectre: disable all speculations
    - High performance overheads!

InvisiSpec

2018 51st Annual IEEE/ACM International Symposium on Microarchitecture

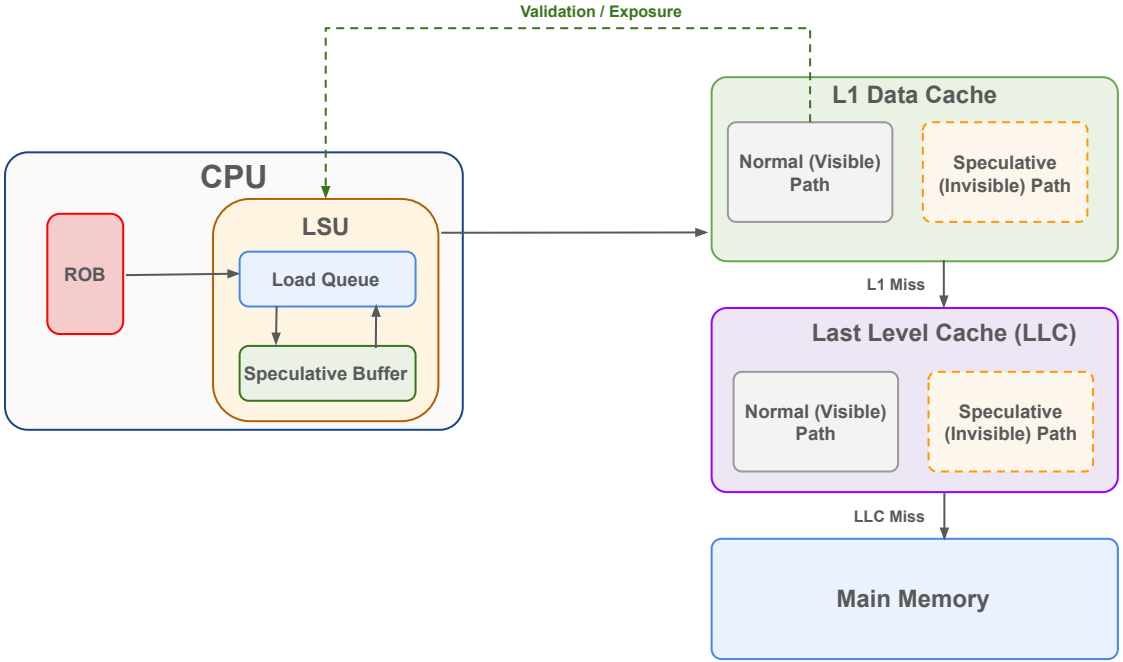
- **Unsafe Speculative loads (USL) invisible** in the data cache hierarchy until it become safe.

- Provide two models
  - V1 - protect against **Spectre-like** attacks based on **branch speculation**.
  - V2 - protect against **futuristic attacks** where **any speculative load** may pose a threat.

- We implemented model-V2.

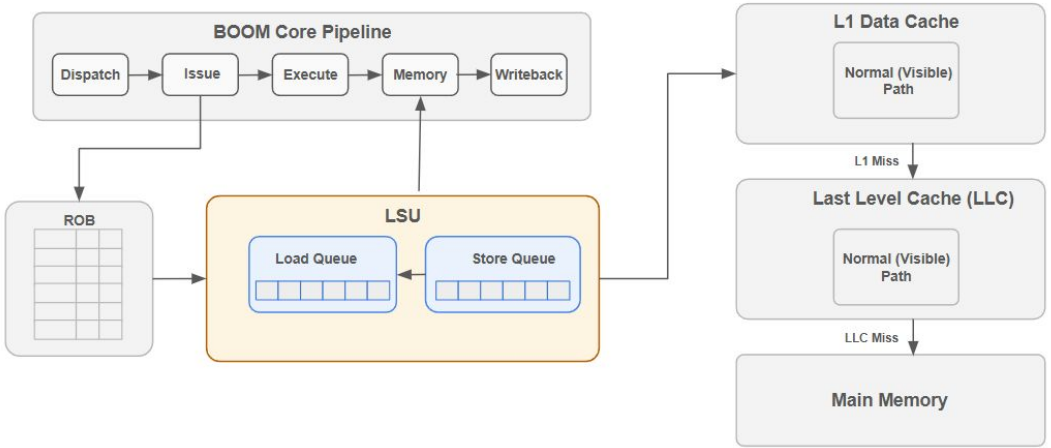
- USLs read data into a **speculative buffer**, without modifying the cache hierarchy.
- When the loads become safe, InvisiSpec makes them visible to the rest of the system.

InvisiSpec Architecture



BOOM Load operation

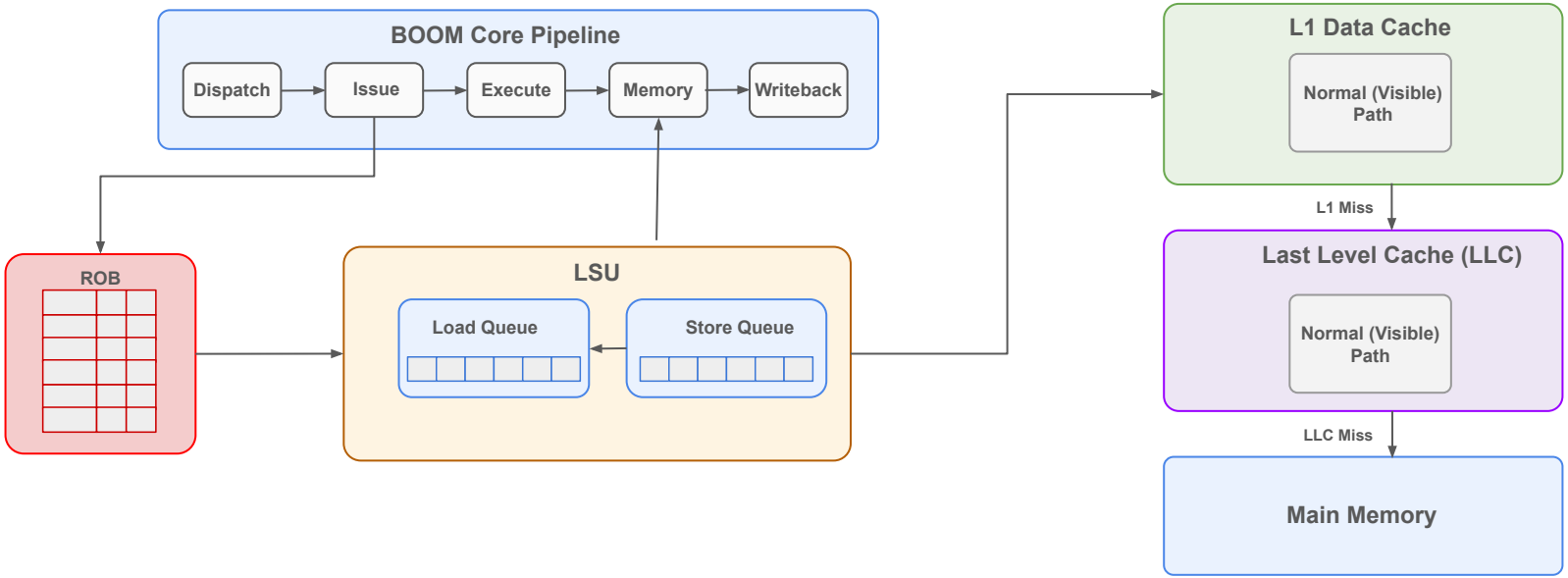
1. LSU put the request in the load queue.
2. L1 DCache Lookup
3. If cache hit:
   - Load data from L1 cache.
4. If cache miss:
   - Allocate MSHR in L1 cache.
   - Send Acquire to L2 cache.
   - Refill cache line.
   - Update cache metadata and data arrays.



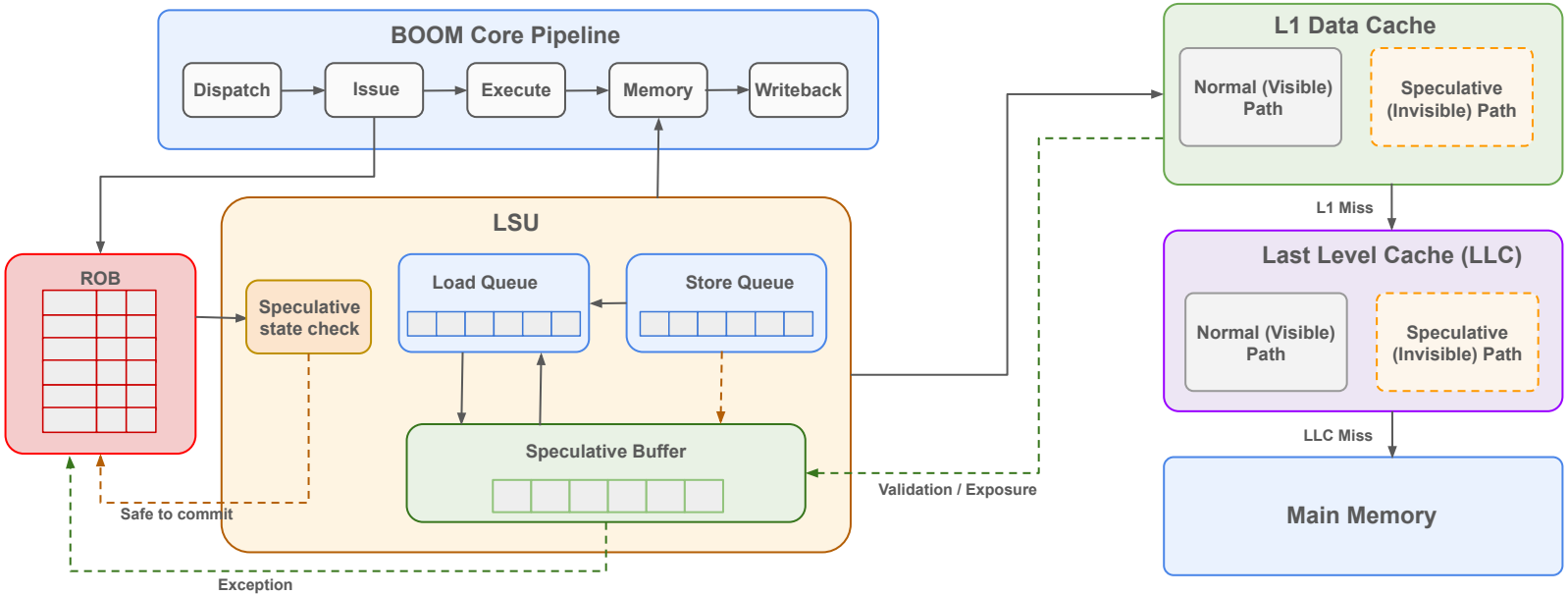
All the loads including speculative loads are treated in the same way.

- **Transient loads will change the cache state.**
- **Side channel attacks are possible.**

BOOM Implementation



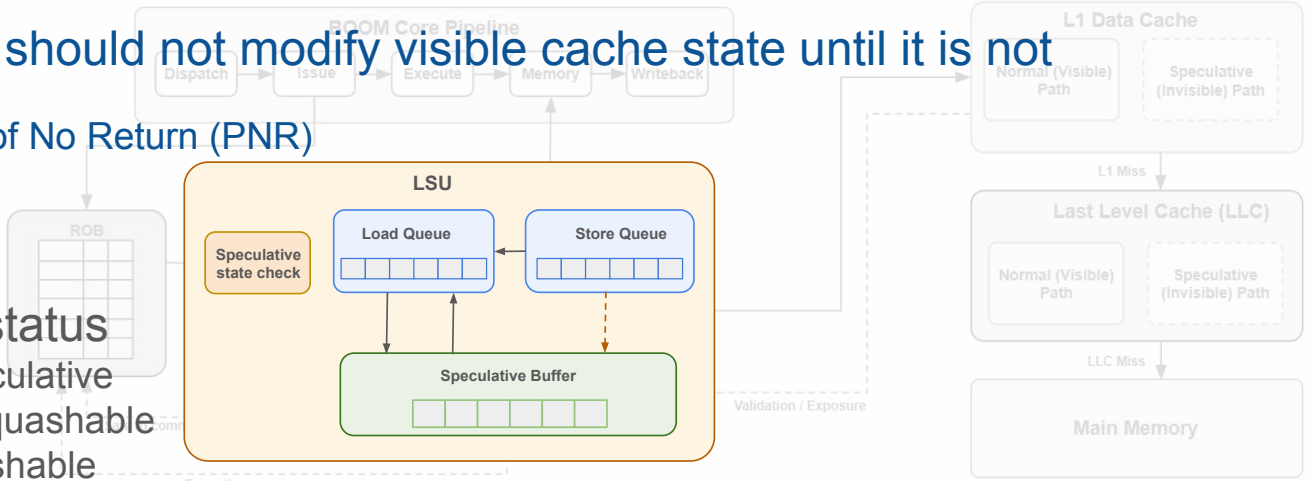
InvisiBOOM Implementation



InvisiBOOM Implementation

Main idea:

- Speculative loads should not modify visible cache state until it is not speculative
  - Reach the Point of No Return (PNR)



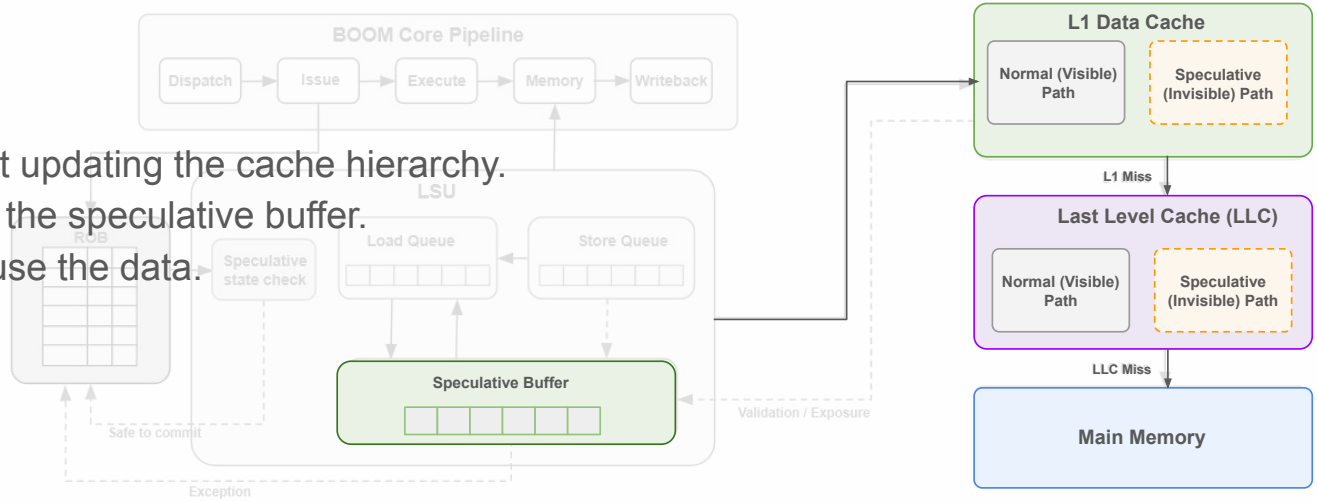
Key concepts:

- Speculative load status
  - Normal - not speculative
  - Exposure - not squashable
  - Validation - squashable
    - Validation / Exposure : depends on the memory model (TSO / RC / RVWMO)
- Speculative Buffer
  - Hold the data of speculative loads.
  - One-to-one mapping with load queue entries.

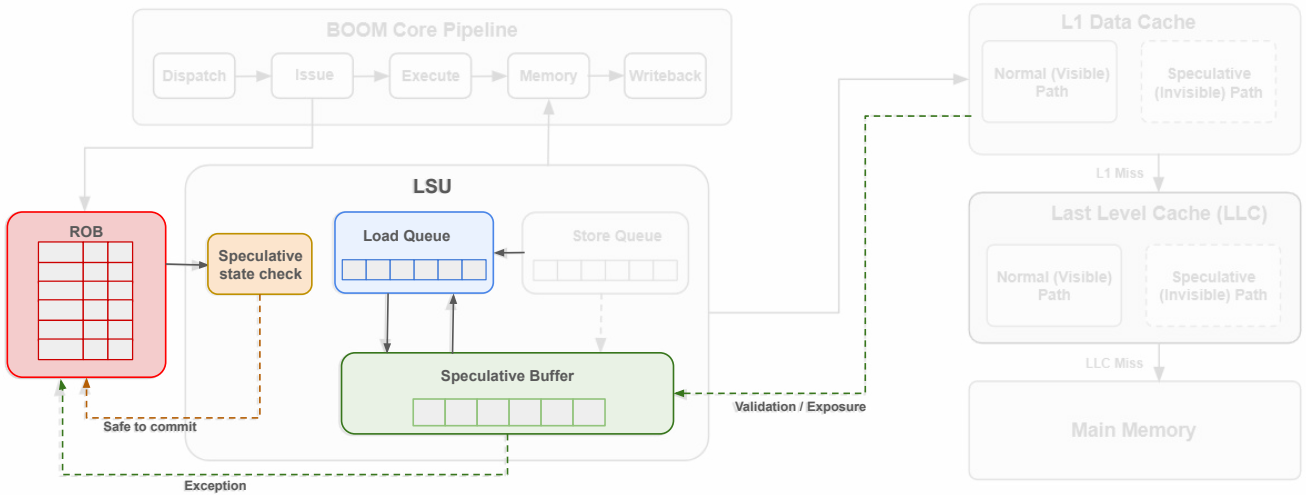
InvisiBOOM Implementation

Key concepts:

- Speculative loads
  - Load data without updating the cache hierarchy.
  - Data is loaded to the speculative buffer.
  - Functional units use the data.



InvisiBOOM Implementation



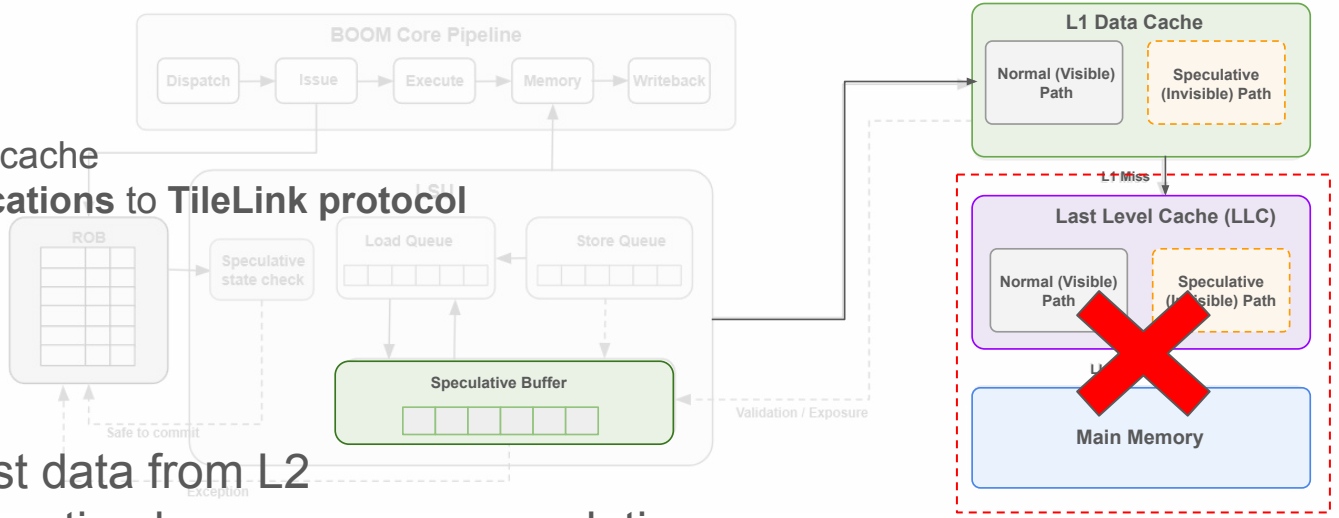
Key concepts:

- Visibility point
  - ROB PNR
- Load Exposure
  - Let the cache hierarchy know about the data
- Load Validation
  - Compare speculatively loaded data with latest data
  - Data match: clear_unsafe & commit
  - Data mismatch: mini_exception & ROB flush

RPTU

Speculative Miss Handling in L1 Cache

- Ideally:
  - Request from L2 cache
  - But need **modifications** to **TileLink protocol**



- Current policy:
  - Do not request data from L2
  - Wait until instruction becomes non-speculative
  - Then perform normal memory access.

Results

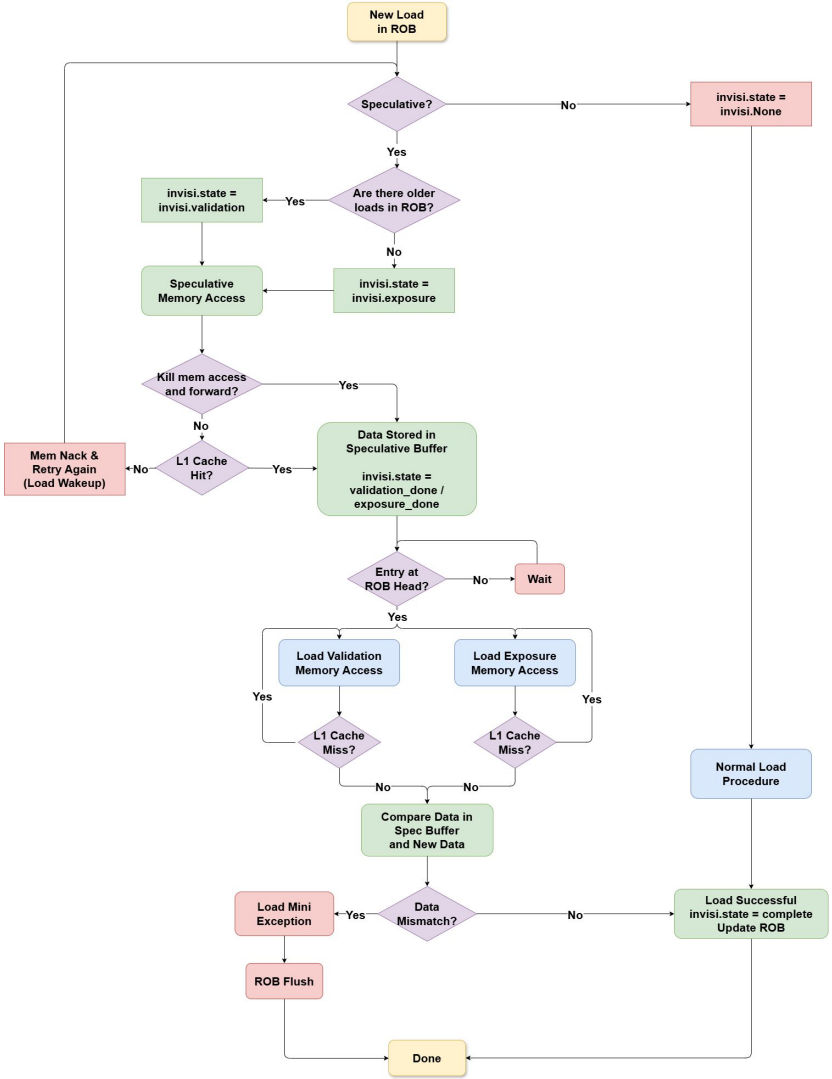
- Successfully run *riscv-tests/benchmarks.*
- Successfully run *riscv-torture* tests.
  - Randomized stress-test generator
- Linux Booting is not yet successful.
  - Kernel panic error
  - [TODO] Test on the latest version

Future Optimizations

- Handle L1 cache misses for speculative loads
- Optimize by reducing the number of validations
  - Transform validations to exposures based on the memory policy
- Adversarial receiver and victim program could reside in
  - Same thread - currently supported
  - Cross Core - currently not supported
  - SMT - BOOM does not support

Thank you!

Current Implementation



Other solutions for Spectral Attacks

- Kiriansky et al. — SafeSpec: Banishing the Spectre of a Meltdown with Leakage-Free Speculation (ISCA, 2018)

- Hennessy et al. — InvisiSpec: Making Speculative Execution Invisible in the Cache Hierarchy (MICRO, 2018)

- Wang et al. — DAWG: A Defense Against Cache Timing Attacks in Speculative Execution Processors (MICRO, 2018)